

Chapter Two

Linked List

Course Outline



❖ ADT

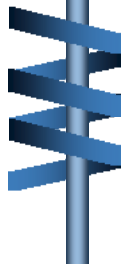
❖ Basics of pointer

❖ Linked List

❖ Operations in Linked list

❖ Linked list variations

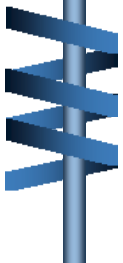
❖ Stack and Queue



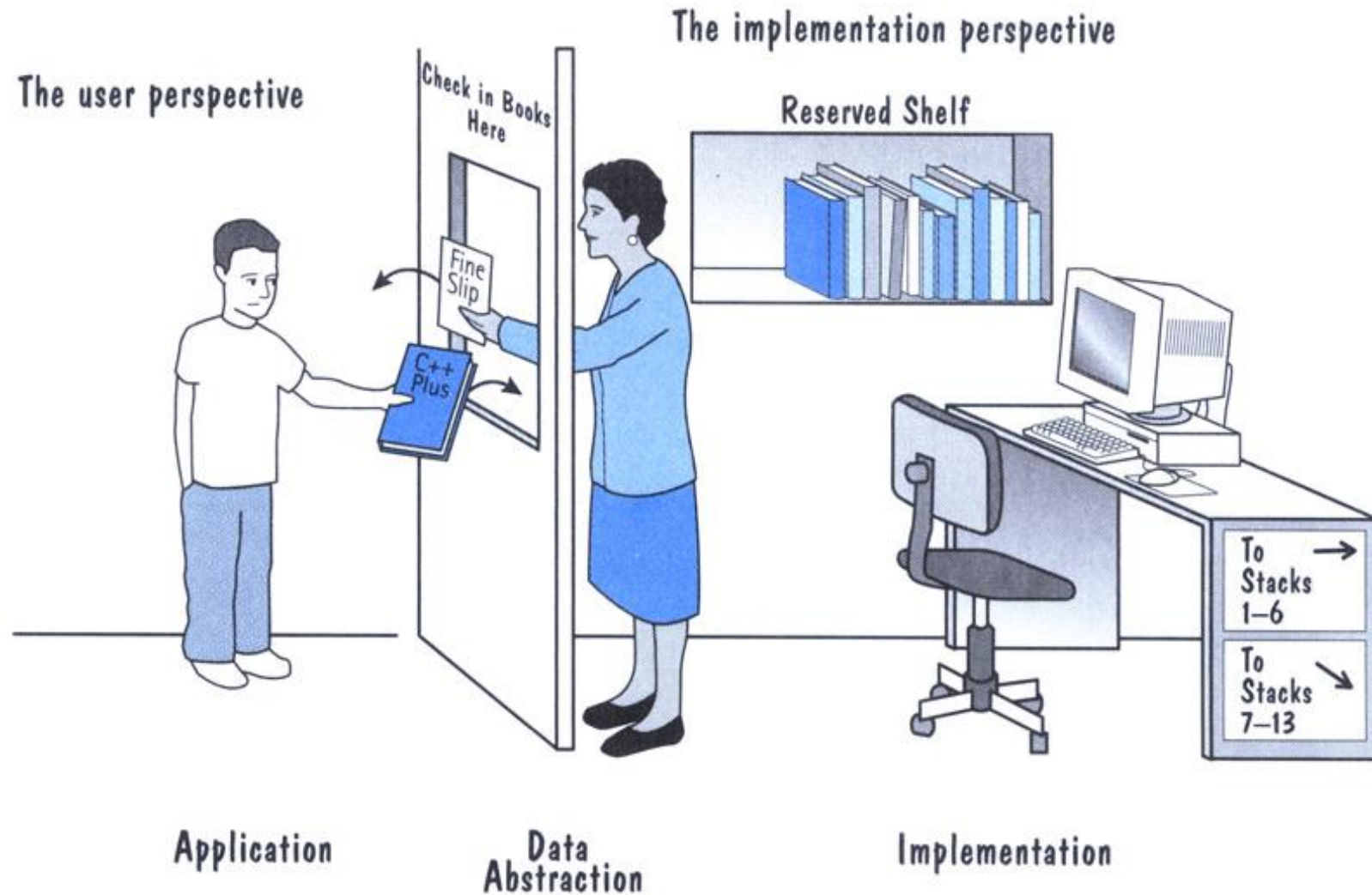
Data abstraction



- Usually, the information that is available to the computer consists of a selected set of data about the actual problem
- The data represent an **abstraction** of reality in the sense that certain properties and characteristics of the real objects are ignored because they are peripheral and irrelevant to the particular problem
 - **abstraction** is simplification of facts
- **Example:** Consider a personnel file of an employer. Every employee is represented (abstracted) on this file by a set of data relevant to his/her accounting procedures. This set may include Name, Age, Salary but it will most probably not include irrelevant data such as the hair color, weight, and height.



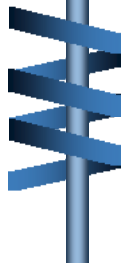
Data abstraction



ADTs (Abstract Data Types)



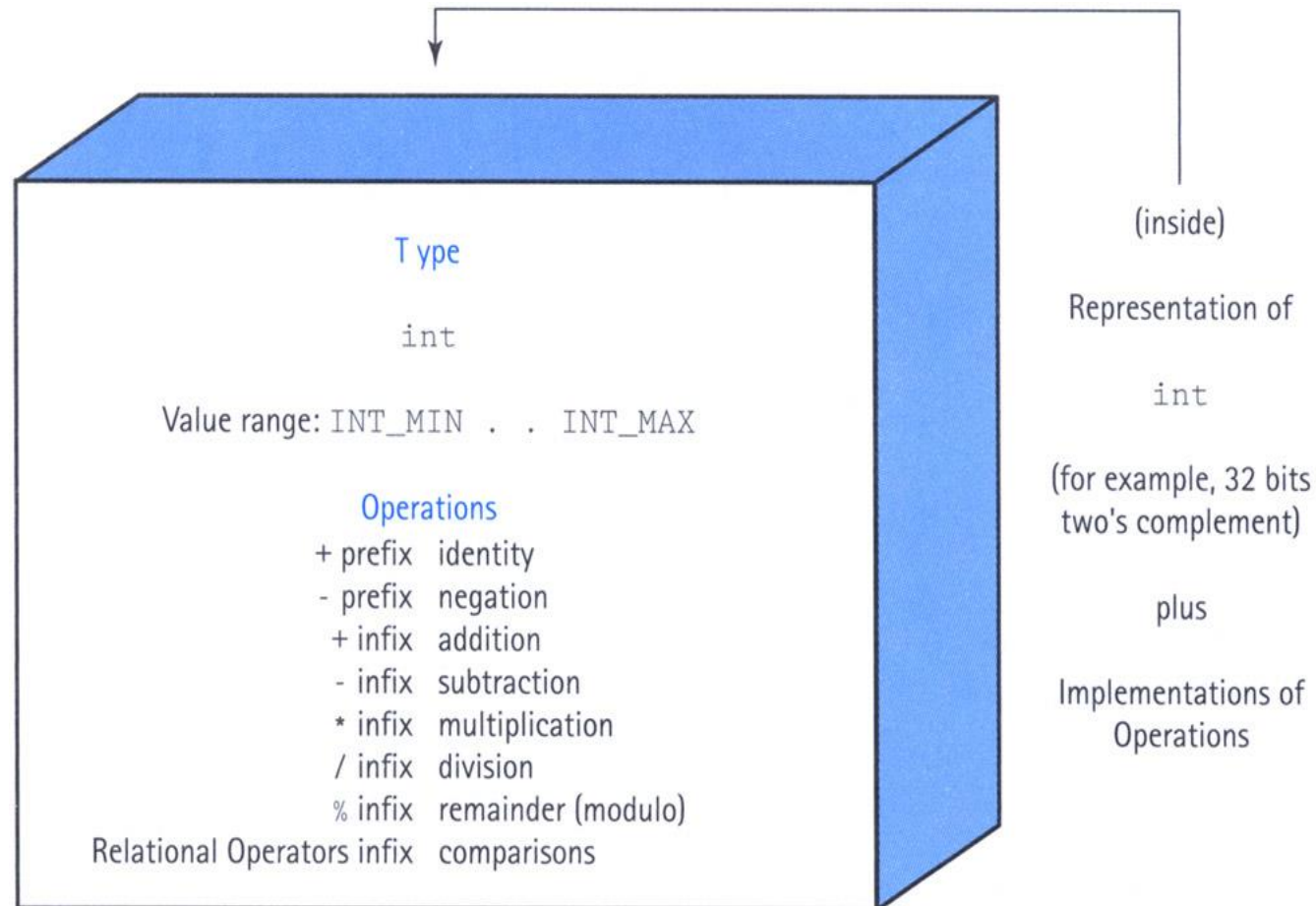
- An Abstract Data Type (ADT) is the realization of a data type as a software component
- An ADT is data type whose properties (domain and operations) are specified independently of any particular implementation
- An ADT does not specify how the data type is implemented
- Implementation details are hidden from the user of the ADT and protected from outside access, a concept referred to as **encapsulation**.



ADTs (Abstract Data Types)



- The built-in data types themselves have their own ADT representations. Consider the *int* type

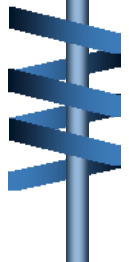


ADTs (Abstract Data Types)



Example : **Operating a car**

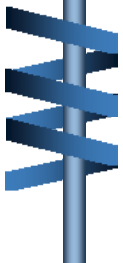
- Primary activities are steering, accelerating, and braking.
 - ✓ Steer by turning the steering wheel
 - ✓ Accelerate by pushing the gas pedal
 - ✓ Brake by pushing the brake pedal
- An ADT with operations **steer**, **accelerate**, and **brake**.
- Two cars might implement these operations in different ways:
 - ✓ with different types of **engine**
 - ✓ front- versus rear-wheel drive
- Most drivers can operate many different cars because the ADT presents a uniform method of operation



Basics of Pointers



- Every **byte** in the computer's memory has an *address*.
- A variable that holds an address value is called a *pointer variable*, or simply a *pointer*
- What is the data type of pointer variables?
 - ✓ `int* ptr;`
 - ✓ the **ptr** variable is not type *int*, the statement just defines a pointer variable that points to integer types or holds the address of integer types
- What's wrong with the idea of a general-purpose pointer type that holds pointers to any data type?
 - the compiler needs to know *what kind of variable the pointer points to* (for purpose of address calculation at run time)



Basics of Pointers



- Accessing values and pointer variables

- Accessing the value of the pointer

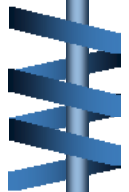
```
int* ptr;  
int x=10;  
ptr=&x;  
cout<<ptr;    //prints the value contained by ptr
```

Example: **0x29cf44**

- Accessing the value pointed to by the pointer (by the value of the pointer)

```
Cout<<*ptr;
```

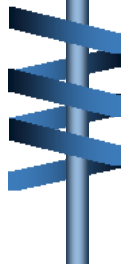
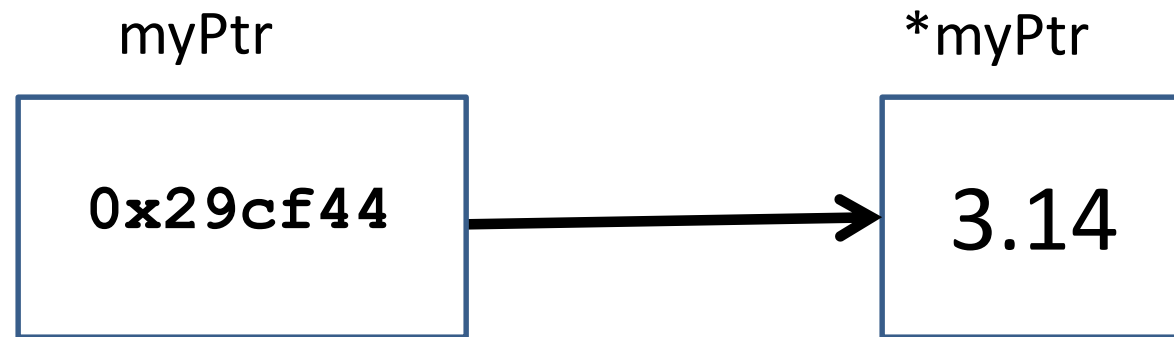
Output: **10**



Basics of Pointers



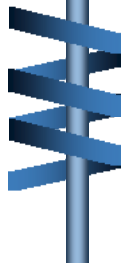
```
double* myPtr;  
double a=3.14;  
myPtr=&a; //addressof(a)
```



Basics of Pointers



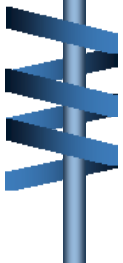
- **What are pointers for?**
 - Accessing array elements
 - Passing arguments to a function when the function needs to modify the original argument (passing by reference)
 - Passing arrays and strings to functions
 - Obtaining memory from the system
 - Creating data structures such as *linked lists*



Lists



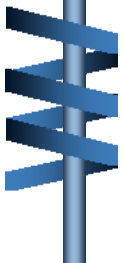
- A list is a finite sequence of data items known as *elements*.
- Properties of lists
 - ✓ Position of elements
 - ✓ Length of the list
 - ✓ Beginning of the list
 - ✓ End of the list
 - ✓ Sorted
 - ✓ Unsorted



Lists (array implementation)



- An array is a kind of list where the elements are accessed by
 - identifier
 - index
- Static memory is allocated to an array of values
- The allocated memory will be reserved to that array even if it is not used
 - This, in fact leads to memory wastage. What if you want the memory grow only when an element is added?



Linked list (Pointer implementation)

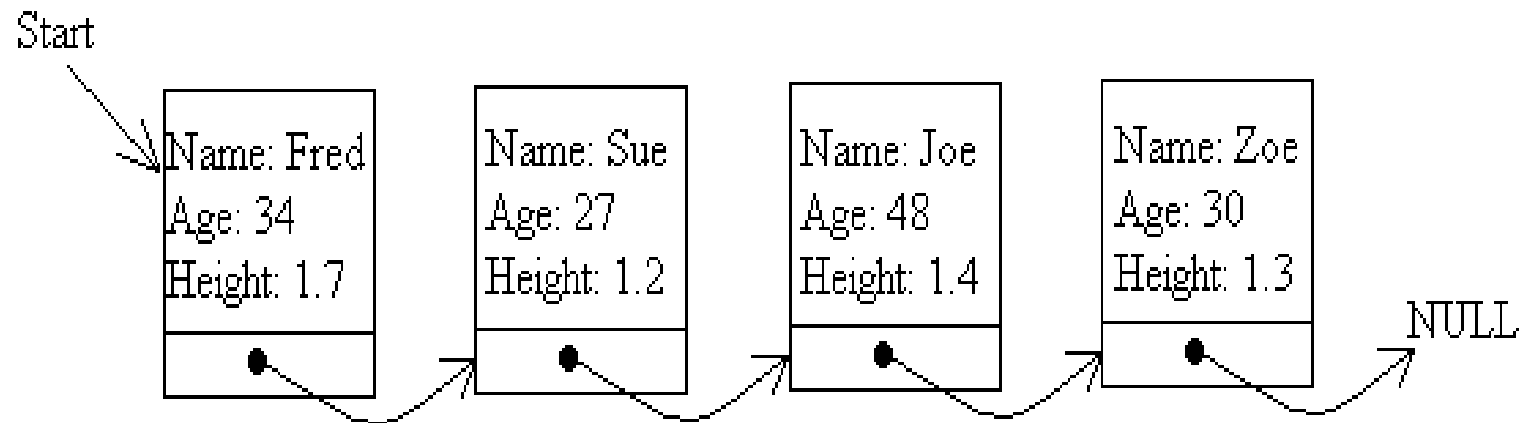
- is a technique of creating a list with the ability to **add** and **delete** items at run time (unlike arrays which allocate memory statically)
- is a data structure that is built from **structures** and **pointers**.
- is made up of cells, each cell consisting of data of the list item and a pointer to the next cell on the list
- If the list is $a_1, a_2, \dots, a_n$, *the cell holding a_i has a pointer to the cell holding a_{i+1}*
- is made up of a chain of **nodes** where each node contains:
 - ✓ the data item, and
 - ✓ a pointer to the next node

Linked list



```
struct list {  
    char name[10];  
    int age;  
    float height;  
    struct list *next;  
};
```

- A linked list based on the above ADT can be represented by a diagram:



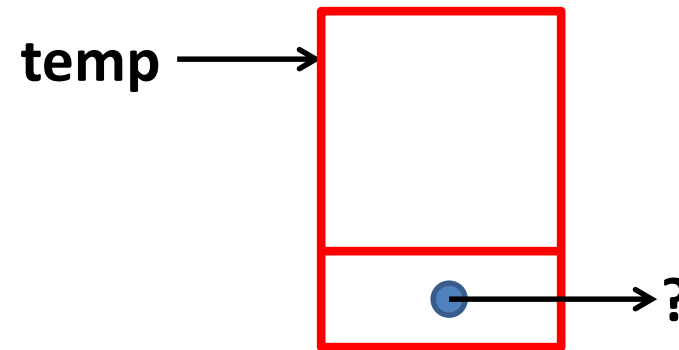
Linked list

- If we have to define a data structure for a linked list

```
struct list {  
    char name[10];  
    int age;  
    struct list *next;  
};  
list *start = NULL;
```

- Declaring a list type

```
list *temp = new list;
```



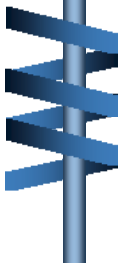
Linked list



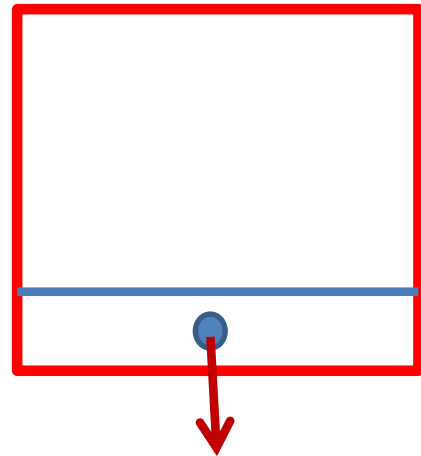
- Creating the first object (element of the list)

```
cout<<"Enter name: ";  
cin>>temp->name;  
cout<<"Enter age: ";  
cin>>temp->age;  
cout<<"Enter height: ";  
cin>>temp->height;  
start=temp;  
temp->nxt=NULL;
```

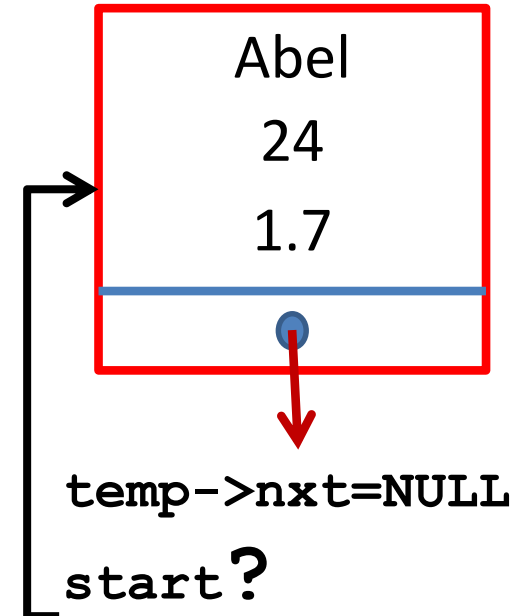
- The last line sets the pointer from this node to the next to NULL



Linked list



start=NULL



temp->nxt=NULL

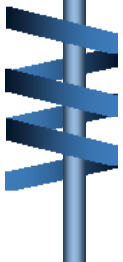
start?

- After creating the first list element, the *start* pointer will point to this node (*temp*)
- The *nxt* pointer of this node will be **NULL** because there is no another list element to point to

Operations in Linked list



- Insertion
 - at the beginning
 - at the end
 - after specified number of nodes
- Navigation through list items
- Deletion
 - from the beginning
 - from the end
 - a specified node

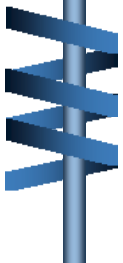


Insertion

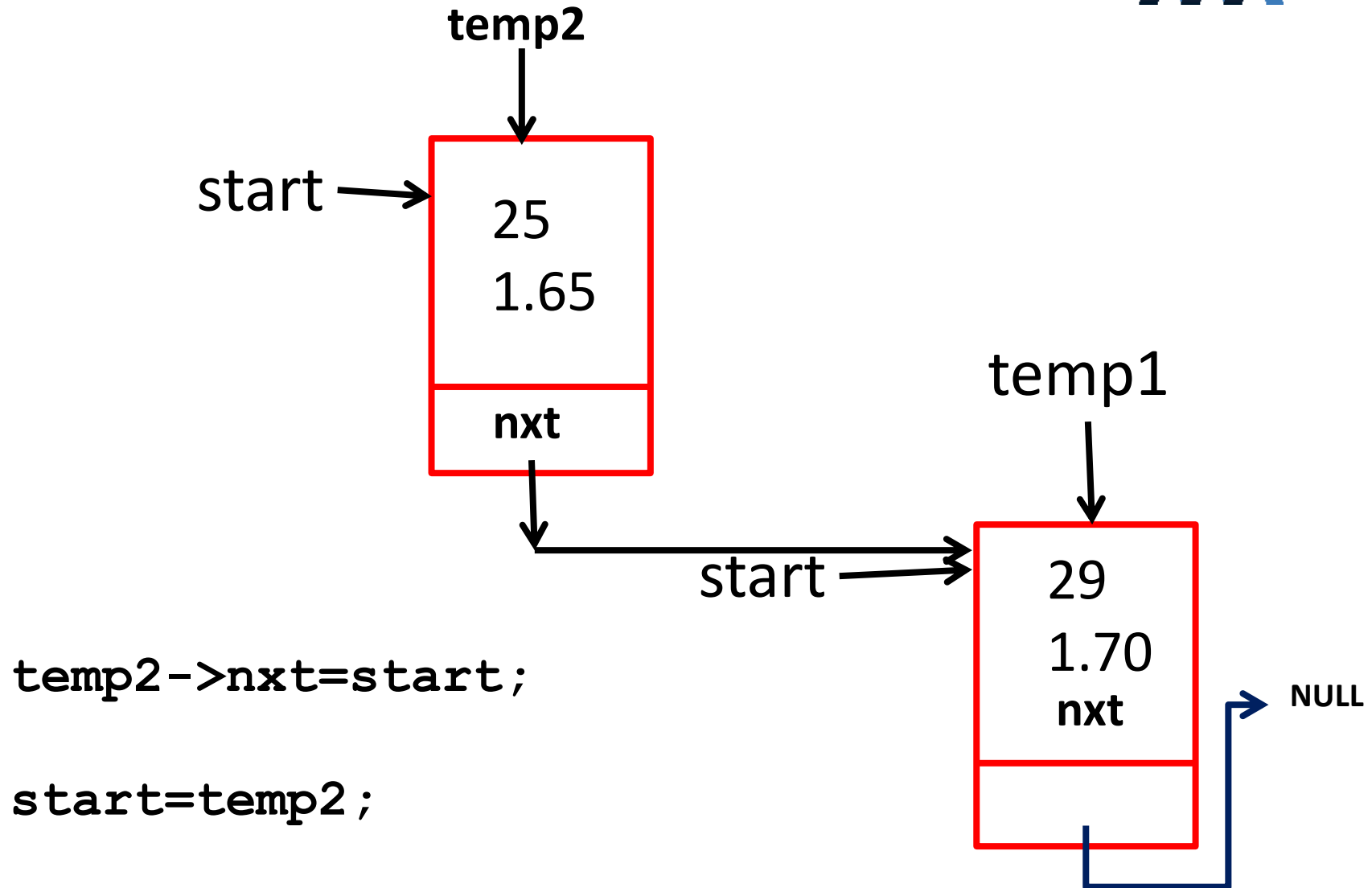


- **At the beginning** (*assuming the list is not empty*)
 - *start* pointer is now pointing to some node
 - At first initialize node type:

```
list *temp=new list; //creating a list  
variable and allocating memory
```
 - Store the required values in *temp*
 - (*temp->name*, *temp->age*, *temp->height*)
 - The *nxt* pointer of *temp* (*temp->nxt*) should hold the address of the element that was at the front.
 - The *start* pointer (which was pointing to the element at the front) should now hold the address of *temp*



Insertion

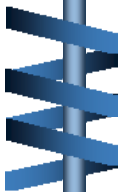


Insertion



```
struct list{  
    int age;  
    float height;  
    list *nxt;  
};  
list *start = NULL;
```

```
int main() {  
    list *temp1=new list;  
    temp1->age=29;  
    temp1->height=1.70;  
    temp1->nxt=NULL;  
    start=temp1;  
  
    list *temp2=new list;  
    temp2->age=25;  
    temp2->height=1.75;  
    temp2->nxt=start;  
    start=temp2;  
    return 0;  
}
```



Insertion



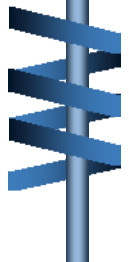
- **At the end**

- Here the first job is to **find the last node** of the given linked list
- First we declare a **temp** variable (*list item*)
- If the list is empty, the **start** pointer is made to point to **temp**

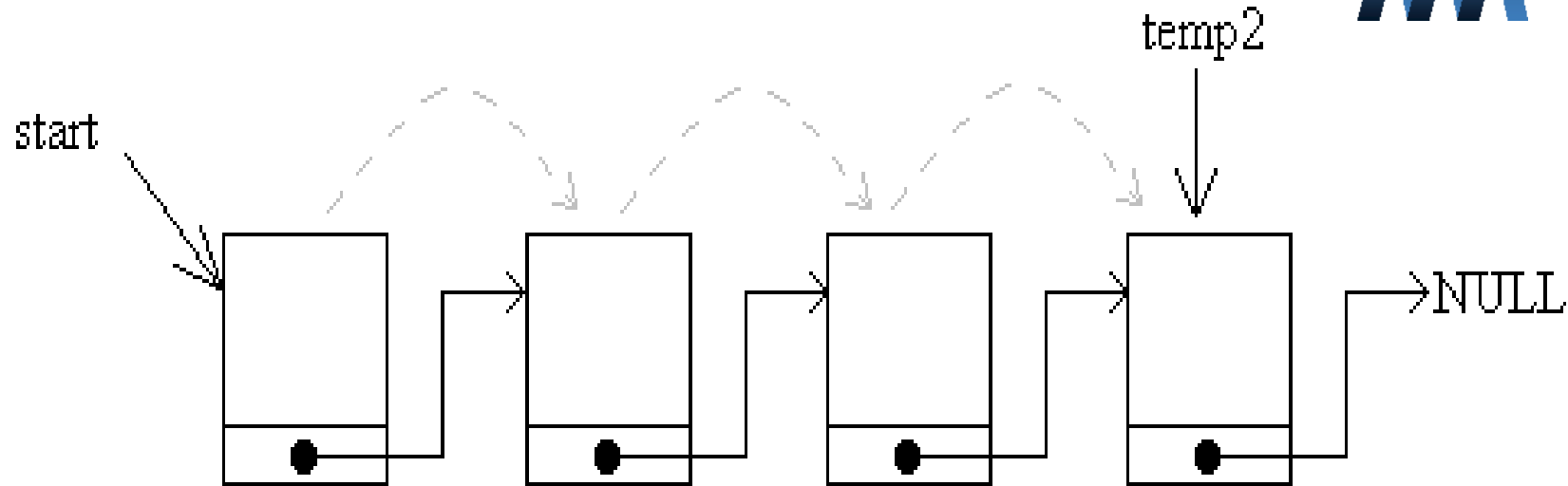
```
if (start==NULL)
```

```
start=temp;
```

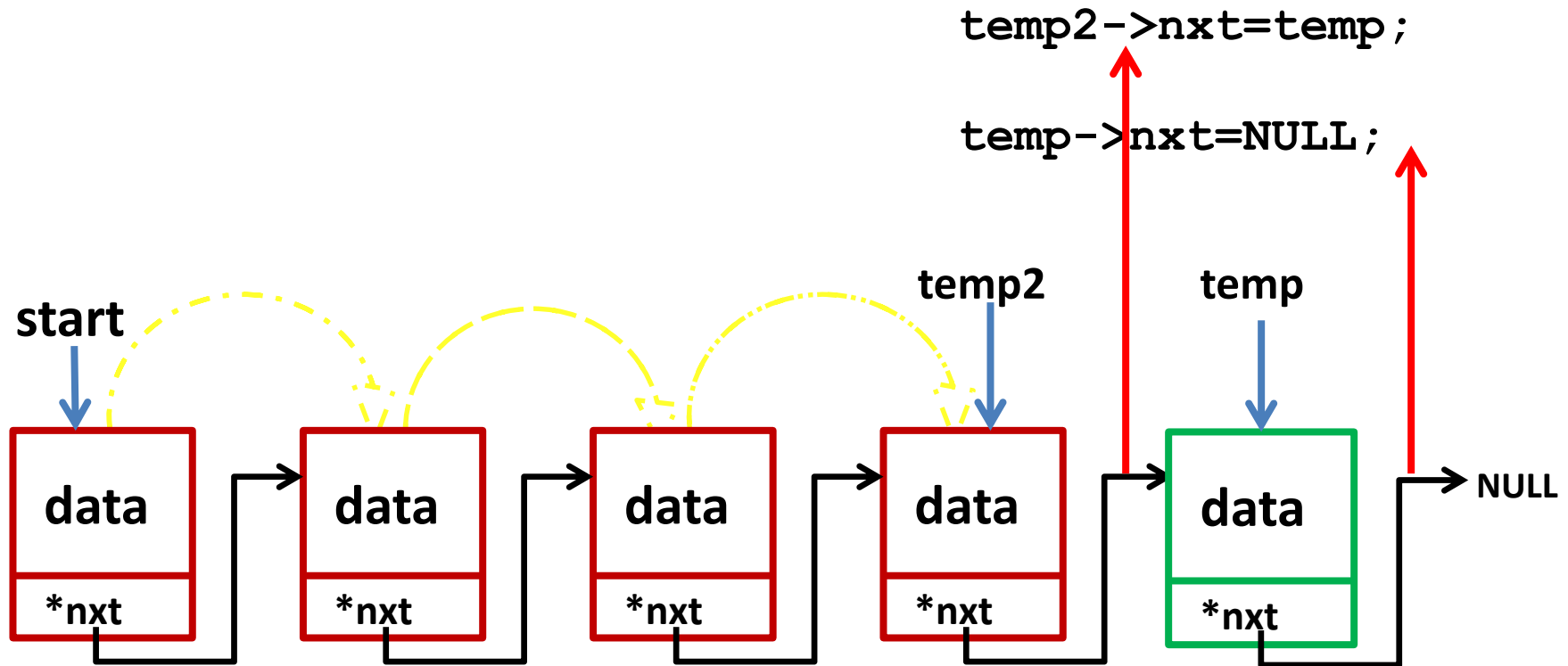
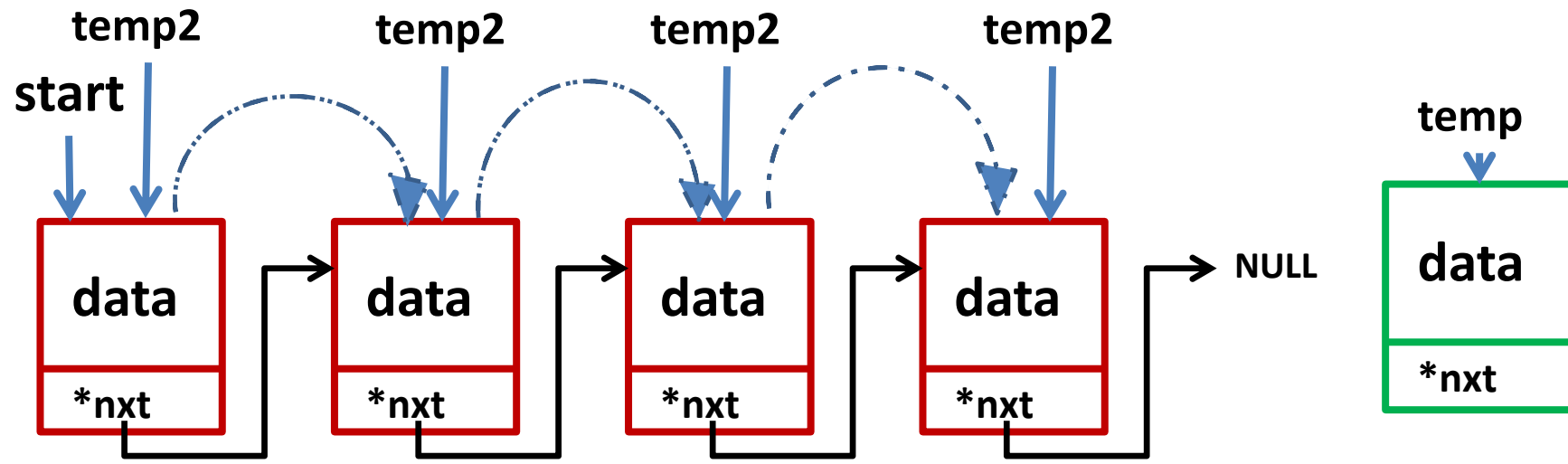
- Otherwise, we declare a second pointer, **temp2**, to step through the list until we find the last node (*the last node is a node whose **nxt** pointer is NULL*)



Insertion: At the end



```
list *temp2 = start; //create a temporary list object
while (temp2->nxt != NULL)
{
    temp2 = temp2->nxt;
}
temp2->nxt=temp; //temp is the node to be inserted
temp->nxt=NULL;
```



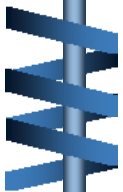
Insertion



- **After specified number of nodes**
 - The first step is to navigate through the specified number of nodes with a temporary node variable, *temp1*

```
node temp1=new node;
temp1=start;
cout<<"Enter node number:";
cin>>node_number;
for(int i=1;i<node_number;i++) {
    temp1 = temp1->next; // go to the next node
    if( temp1 == NULL ) {
        cout<<node_number<<" node is not exist"<< endl;
        break;
    }
}
```

- *temp1* now points to the required position in the list



Insertion: after.....



- The next step is to create the node to be inserted and store the required data

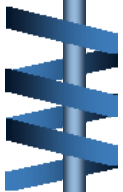
```
node *temp=new node;
```

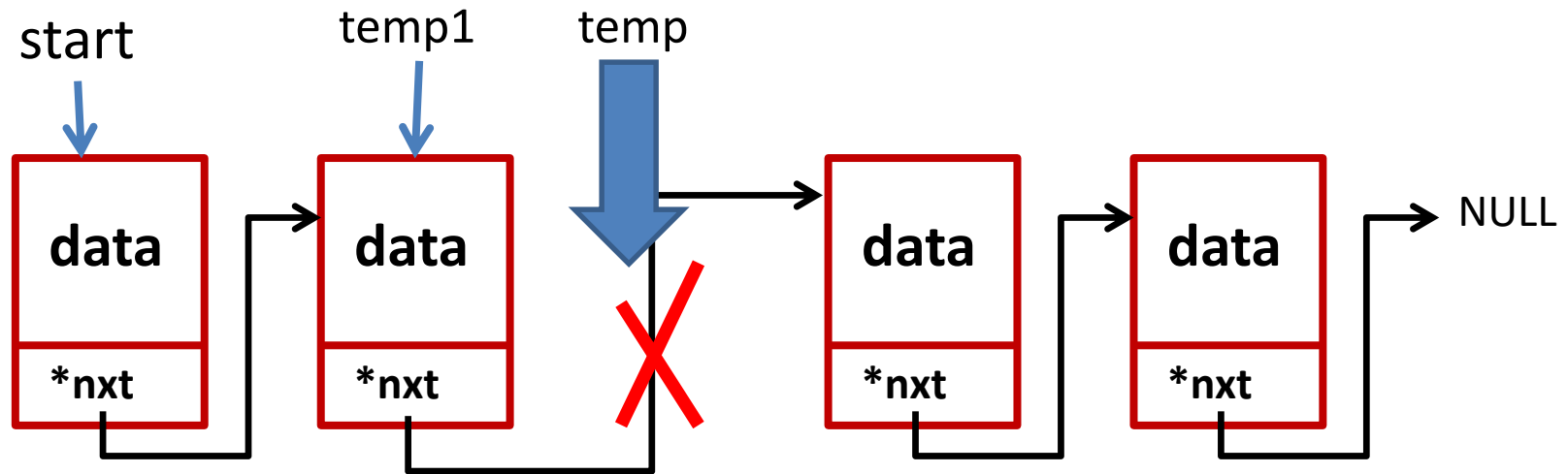
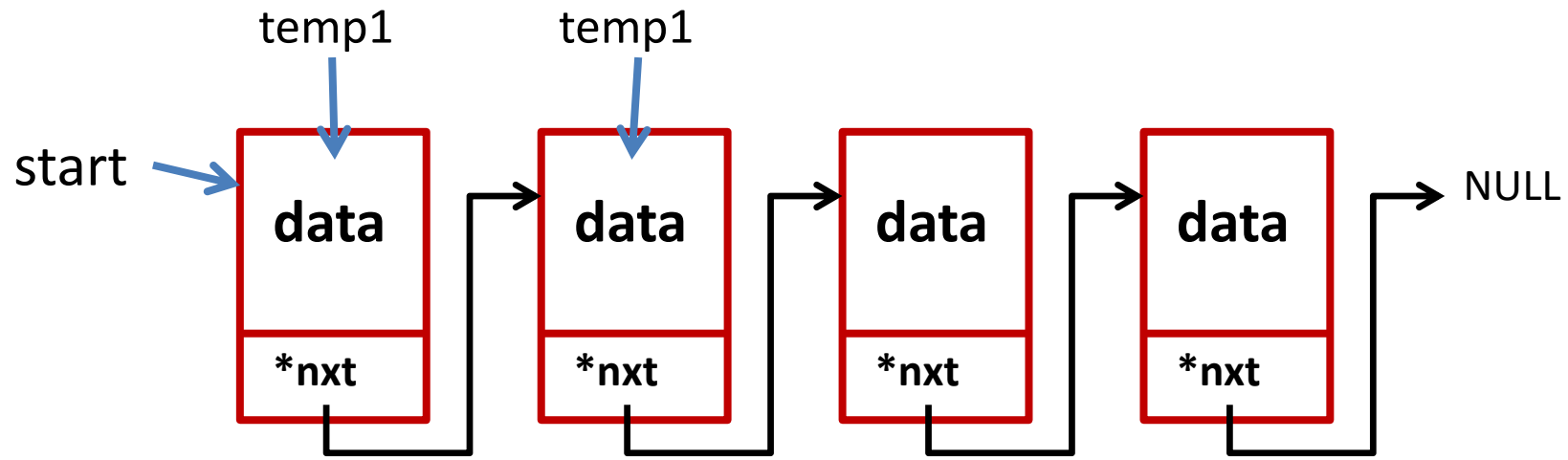
```
temp->data=values;
```

- Last step is to establish the connection between new node and the existing linked list

```
temp->next=temp1->next;
```

```
temp1->next=temp;
```



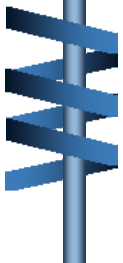


```
temp->next=temp1->next;  
temp1->next=temp;
```

Navigation



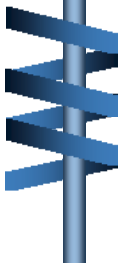
- There may be a need to go through all items in the list for some kind of operation
- Consider the structure of employee with data members Name, Age Salary
- Example:
 - Observe age groups in the list
 - Observe salary distribution
- The implementation of the navigation operation is just the same as the “insert at the end” operation
 - Perform the required operation until NULL pointer is obtained



Navigation ...



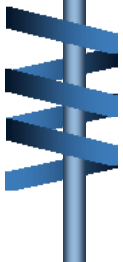
- **Displaying member data of the items in the list**
 1. Set a temporary pointer to point to the same thing as the start pointer.
 2. If the pointer points to **NULL**, display the message "End of list" and stop.
 3. Otherwise, display the details of the node pointed to by the *start* pointer.
 4. Make the temporary pointer point to the same thing as the **next** pointer of the node it is currently indicating.
 5. Jump back to step 2.



Deletion

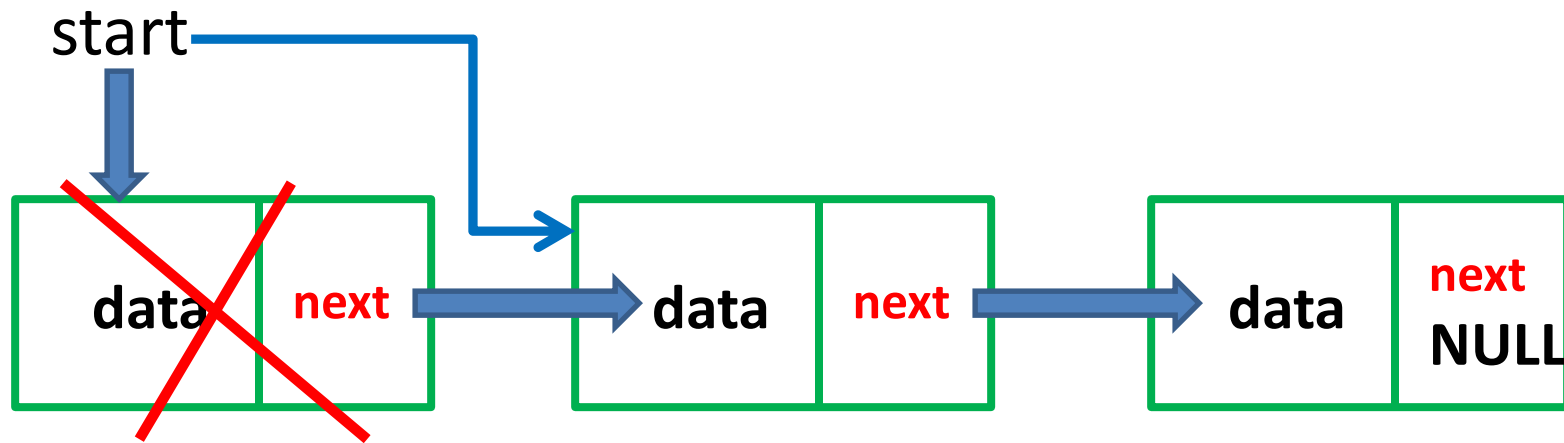


- **From the beginning**
 - Deleting is usually followed by releasing the memory that was allocated.
 - In c++
 - **new** keyword is used to allocate memory
 - **delete** keyword is used to release memory
 - Steps to delete
 1. Create a temporary node called ***temp** and allocate memory
 2. Copy the **start** pointer (head) to **temp**
 3. Transfer the address **temp->next** to **start** pointer (head)
 4. Delete the memory allocated to **temp**



Deletion: from beginning

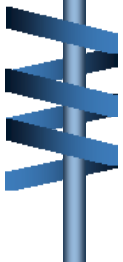
```
node *temp=new node;  
temp=start;  
start=temp->next;  
delete temp;
```



Deletion



- **From the end**
 - The last node's **next** of the linked list always point to **NULL**.
 - So when we delete the last node, the previous node of last node will point to NULL
 - So, we will track ***last node*** and ***previous node*** of the last node in the linked list
 - **next** pointer of ***previous node*** will be made to point to Null
 - Last node will be deleted



Deletion: from the end

- Create temporary nodes called **current** and **previous**

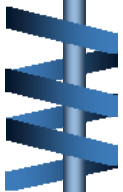
```
node *current=new node;  
node *previous=new node;  
current=start;  
while (current->next!=NULL) {  
    previous=current;  
    current=current->next;  
}  
previous->next=NULL; //making it the last node in the list  
delete current;
```

Deletion



- **A specified node**
 - We need to find the specified node and its previous node

```
node *current=new node;  
node *previous=new node;  
current=start;  
for(int i=1;i<n;i++) {  
    previous=current;  
    current=current->next;  
}  
previous->next=current->next;  
delete current;
```

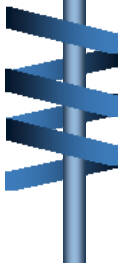


Types Linked list



Linked list variations

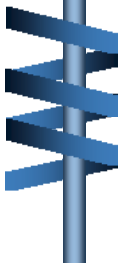
- Singly linked lists
- Circular linked lists
- Doubly linked lists



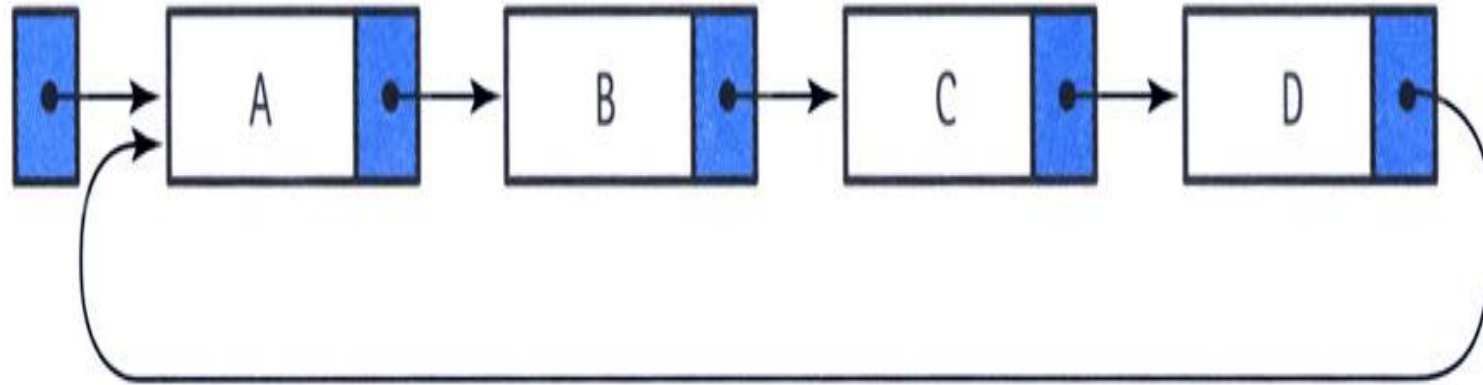
Circular Linked lists



- Linked lists whose last node points to NULL are called linear or open linked lists (singly linked lists)
- In a singly linked list, given a pointer to a node anywhere in the list, we can access all the nodes that follow but none of the nodes that precede it.
- We must always have a pointer to the beginning of the list to be able to access all the nodes in the list.
- We create a circular linked list by making the pointer in the ***next*** member of the **last node** point back to the **first node** instead of containing ***NULL***

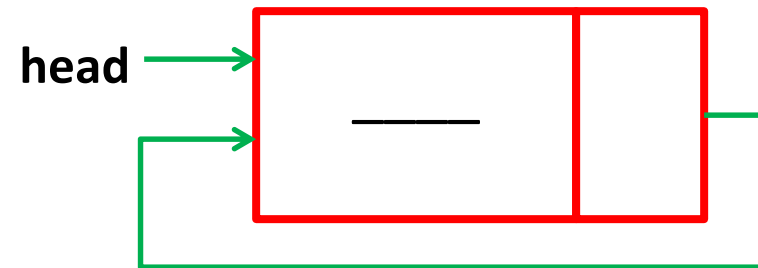


Circular Linked lists



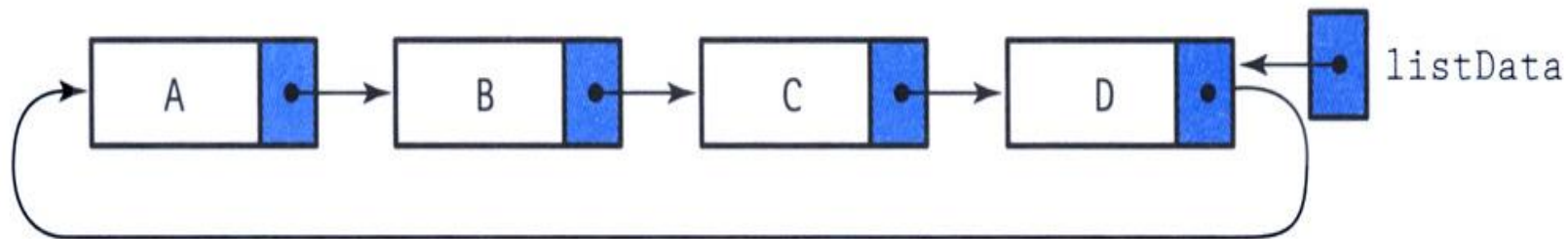
- Structure definition is similar to the singly linked (open)

```
struct node {  
    int age;  
    float height;  
    node *next;  
};  
node *head=NULL;  
head->next=head;
```



Circular Linked lists

- If we let our external pointer point to the last item in the list rather than the first, we have direct access to both the first and the last elements in the list



- To check whether a **current** pointer points to the last node of a circular list, we check for **current->next==head** instead of **current->next==NULL**
- The algorithms insertion into and deletion from a circular list must ensure that the next field of the last node points to the first node of the list upon completion.

Inserting a node to a circular Linked List

- Inserting node at the beginning
- **Step**
 1. Create a temporary pointer **temp** and assign **head** to **temp**

```
node *temp=new node;  
temp=head;
```

2. Navigate through the list until last node is found

```
while (temp->next!=head)  
    temp=temp->next;
```

3. Create a new node to be inserted and load data field

```
node *ND=new node;  
ND->age=20;  
ND->height=1.75;
```

Inserting a node to a circular Linked List

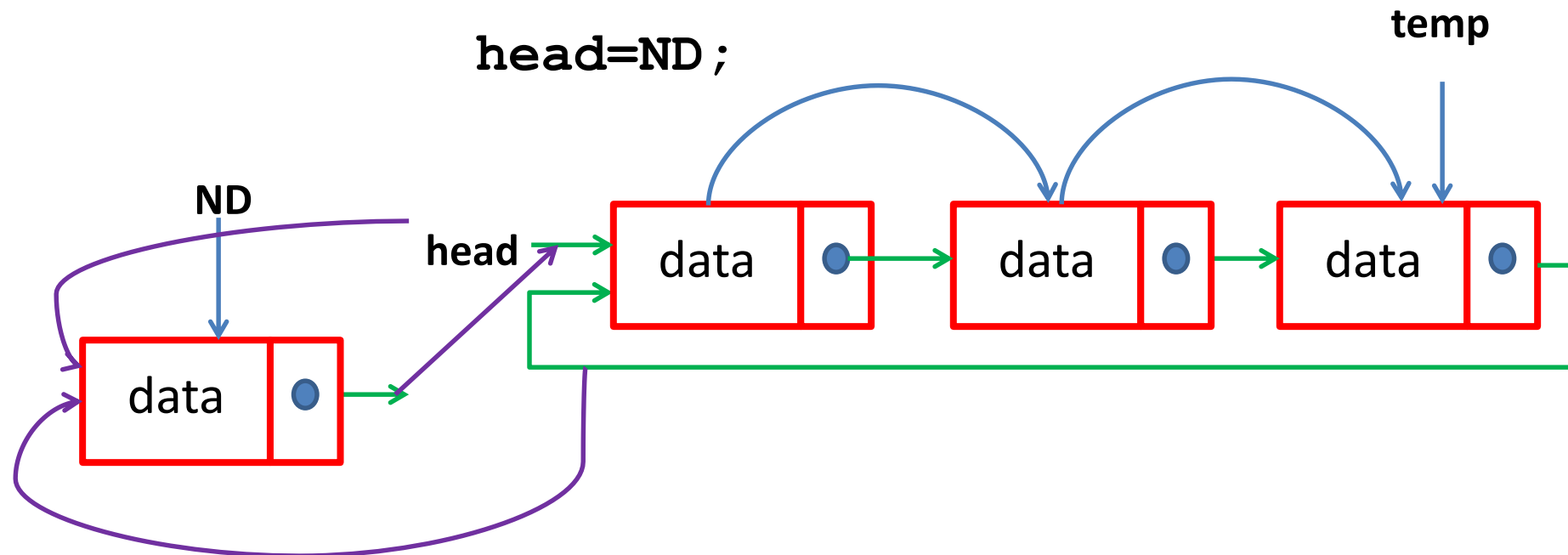
- Inserting node at the beginning

4. Take care of the connection or link between the new node and the existing linked list

`temp->next=ND;`

`ND->next=head;`

`head=ND;`



Inserting a node to a circular Linked List

- Inserting node at the end
- **Step**
 1. Create a temporary pointer **temp** and assign **head** to **temp**

```
node *temp=new node;  
temp=head;
```

2. Navigate through the list until last node is found

```
while (temp->next!=head)  
    temp=temp->next;
```

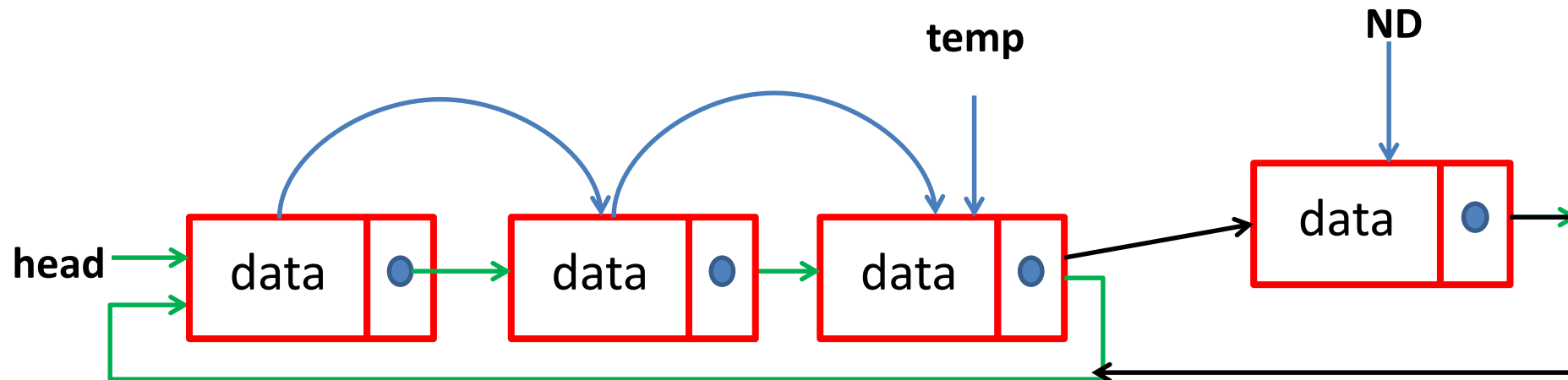
3. Create a new node to be inserted and load data field

```
node *ND=new node;  
ND->age=20;  
ND->height=1.75;
```

Inserting a node to a circular Linked List

- Inserting node at the end
 4. Take care of the connection or link between the new node and the existing linked list

```
ND->next=temp->next;  
temp->next=ND;
```



Inserting a node to a circular Linked List

- **Inserting a node after a specified node**
 - The algorithm for inserting a node after a specified node is the same as that of open linked lists

Deleting a node from a circular Linked List

- **Deleting a node from front**
- The algorithm for deleting a node from front is the same as that of open linked lists

```
node *temp=new node;  
node *temp2=new node;  
temp2=head;  
temp=head;  
while (temp->next!=head)  
    temp=temp->next;  
head=temp2->next;  
temp->next=head;  
delete temp2;
```

Deleting a node from a circular Linked List

- **Deleting a node from back(end)**
- The algorithm for deleting a node at the end is the same as that of open linked lists

```
node *current=new node;  
node *previous=new node;  
current=head;  
while (current->next!=head) {  
    previous=current;  
    current=current->next;  
}  
previous->next=head;  
delete current;
```

Deleting a node from a circular Linked List

- **Deleting a specified node**
 - The algorithm for deleting a specified node in a circular linked list is similar to that of open linked list

Doubly linked lists

- The singly linked list allows for direct access from a list node only to the next node in the list
- A doubly linked list allows convenient access from a list node to the **next** node and also to the **preceding** node on the list
- In a doubly linked list, each node contains a **next-node** link and a **previous-node** in the sequence
- **Application:** traverse a list both forwards and backwards efficiently



Doubly linked lists



- To define a structure to be used as a doubly linked list, in addition to the data, two pointers are included: One for next and one for previous.

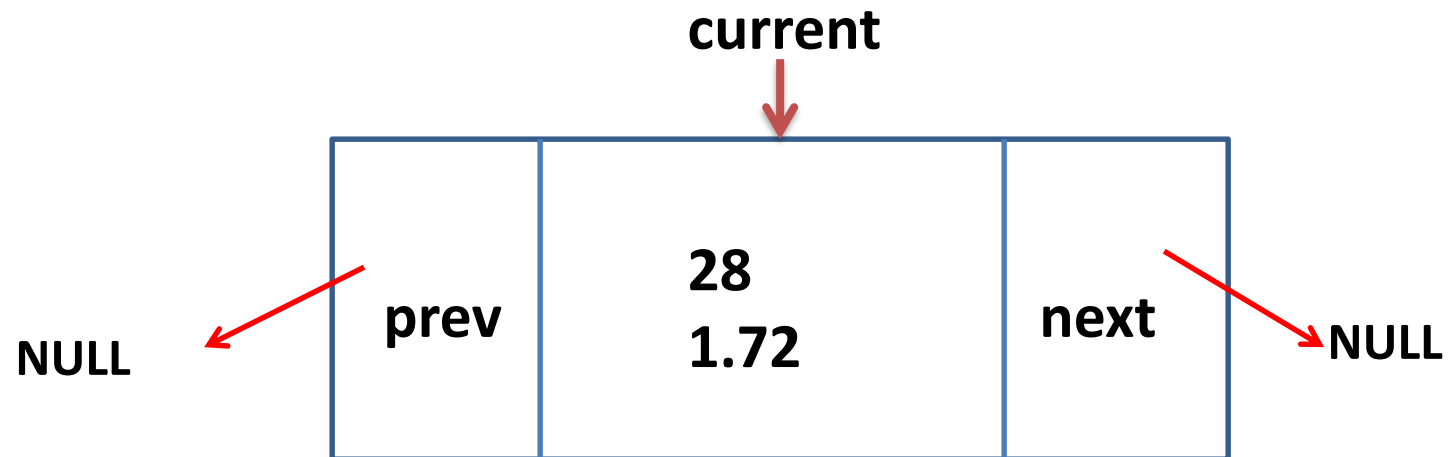
```
struct node {  
    int age;  
    float height;  
    node *next;  
    node *prev;  
};
```

Doubly linked lists



- Let's create the first item of a doubly linked list using the previous structure

```
Node *NODE=new node;  
NODE->age=28;  
NODE->height=1.72;  
NODE->next=NULL;  
NODE->prev=NULL;
```



Inserting a node to a Doubly Linked List

- Inserting node at the beginning

- **Steps**

1. Declare a temporary pointer and move it to *current*

```
node *temp=new node;  
temp=current;
```

2. Navigate to the first node

```
while (temp->prev != NULL)  
    temp = temp->prev;
```

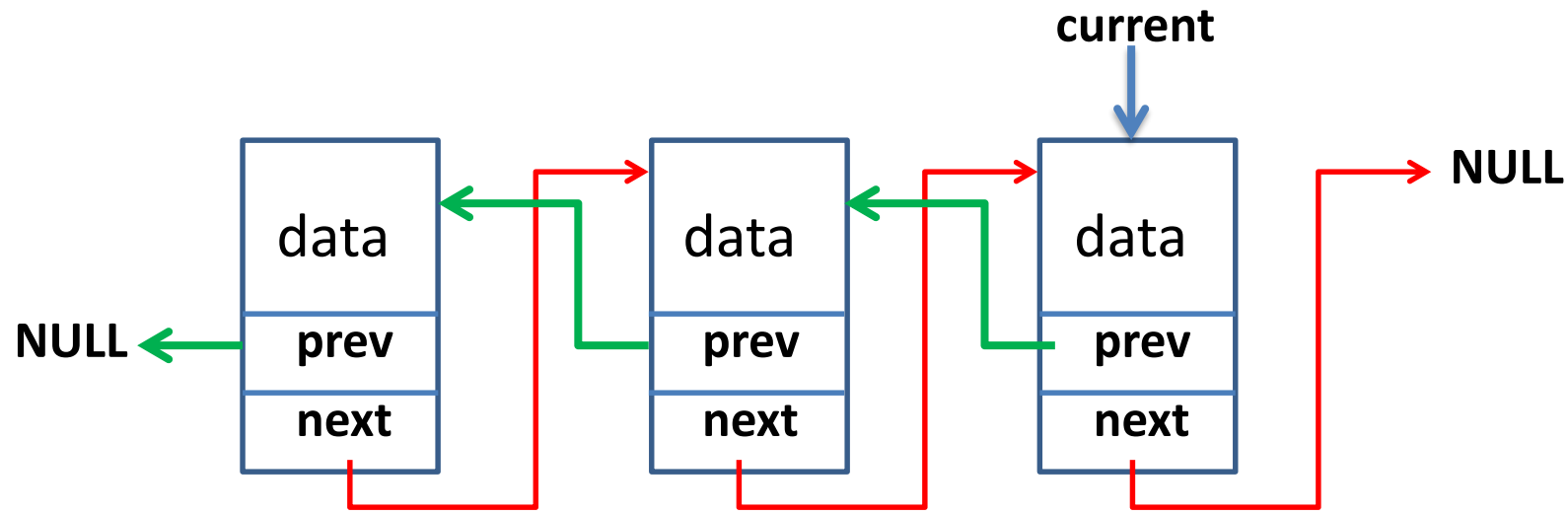
3. Create the item to insert and load the data fields

```
node *temp2=new node;  
temp2->age=23;  
temp2->height=1.8;
```

Inserting a node to a Doubly Linked List

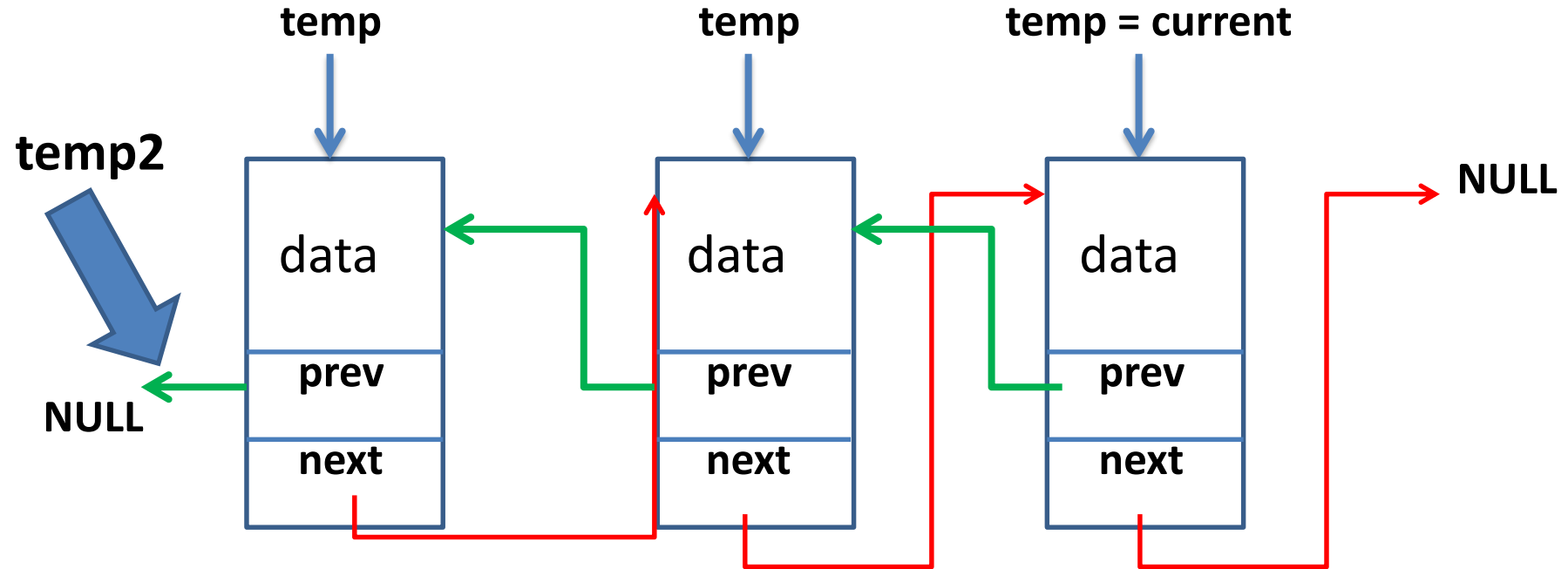
4. Taking care of the link between the new node and existing linked list

```
temp2->prev=NULL; //new start of the list  
temp2->next = temp; // Links to current list  
temp->prev = temp2;
```



```
node *temp=new node;
```

```
temp=current;
```



```
temp2->prev=NULL;
```

```
temp2->next = temp;
```

```
temp->prev = temp2;
```

Inserting a node to a Doubly Linked List

- Inserting node at the end

- **Steps**

1. Declare a temporary pointer and move it to *current*

```
node *temp=new node;  
temp=current;
```

2. Navigate to the last node

```
while (temp->next != NULL)  
    temp = temp->next;
```

3. Create the item to insert and load the data fields

```
node *temp2=new node;  
temp2->age=33;  
temp2->height=1.5;
```

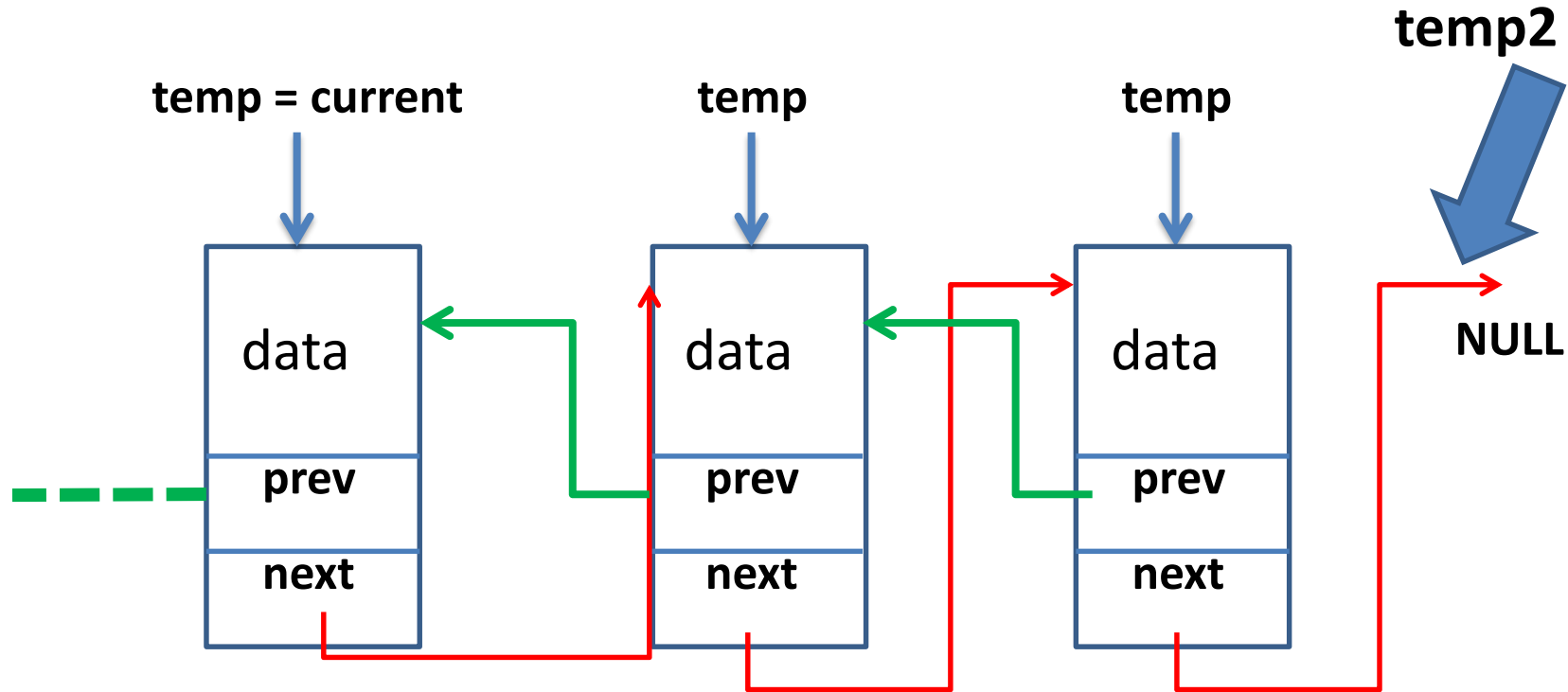
Inserting a node to a Doubly Linked List

4. Taking care of the link between the new node and existing linked list

```
temp2->next=NULL; //new last node  
temp2->prev = temp; // Links to current list  
temp->next = temp2;
```

```
node *temp=new node;
```

```
temp=current;
```



```
temp2->next=NULL;
```

```
temp2->prev = temp;
```

```
temp->next = temp2;
```

Inserting a node to a Doubly Linked List

- Inserting after a specified node

- **Steps**

1. Declare a temporary pointer **CR**

```
CR = head;
```

2. Navigate to the specified node

```
for(int i=1;i<n;i++){  
    CR = CR->next;  
}
```

3. Create the item to insert and load the data fields

```
node *ND = new node;  
ND->age = 33;  
ND->height = 1.5;
```

Inserting a node to a Doubly Linked List

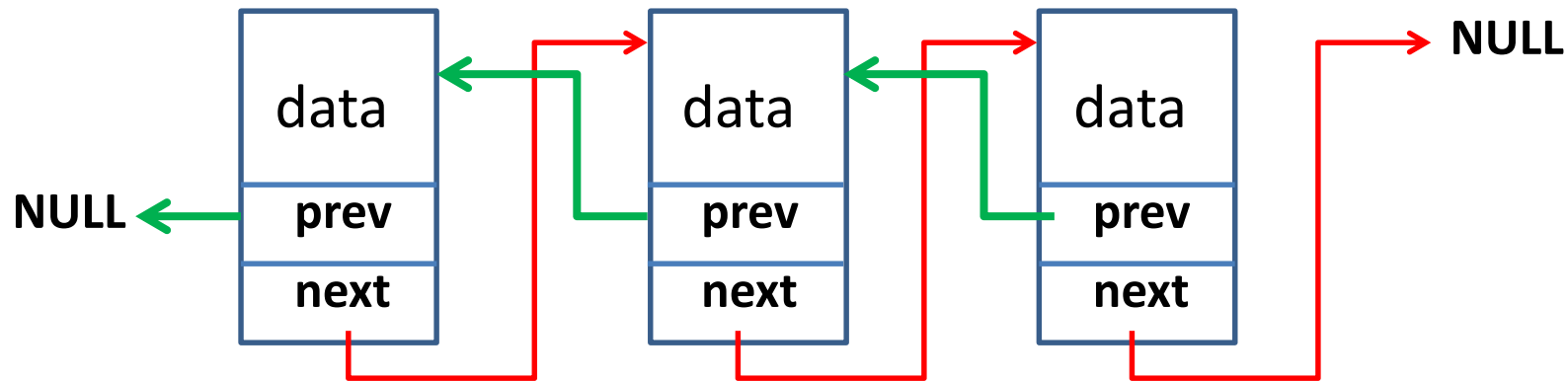
4. Taking care of the link between the new node and existing linked list

```
ND->next = CR->next;  
CR->next->prev = ND;  
CR->next = ND;  
ND->prev = CR;
```

Deleting a node from a doubly Linked List

- Deleting a node from front

```
node *temp=new node;  
temp=head;  
head=temp->next;  
temp->next->prev=NULL;  
delete temp;
```



Deleting a node from a doubly Linked List

- Deleting a node from back(end)

```
node *CR=new node;  
node *PR=new node;  
CR=head;  
while (CR->next!=NULL) {  
    PR=CR;  
    CR=CR->next;  
}  
PR->next=NULL;  
delete CR;
```

Deleting a node from a doubly Linked List

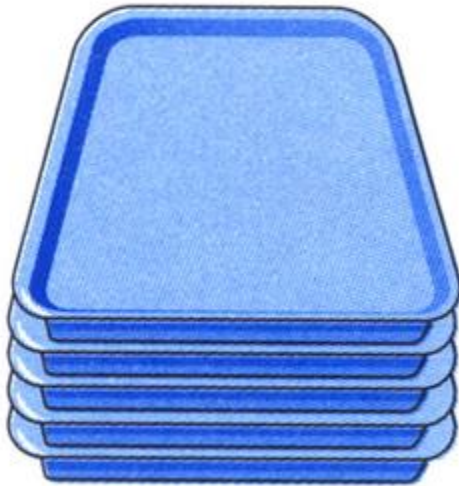
- Deleting a specified node

```
node *CR=new node;  
CR=head;  
for(int i=1;i<n;i++)  
    CR=CR->next;  
CR->prev->next=CR->next;  
CR->next->prev=CR->prev;  
delete CR;
```

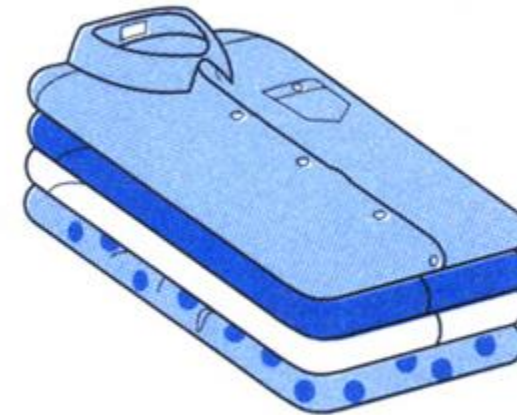
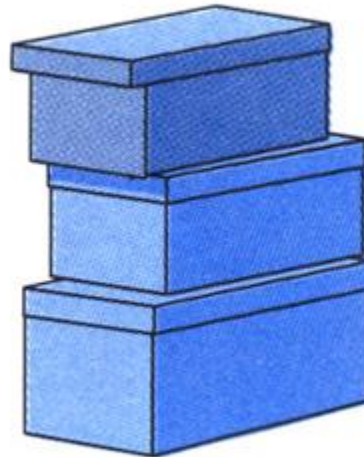
Stack



A stack of
cafeteria trays



A stack of
shoe boxes



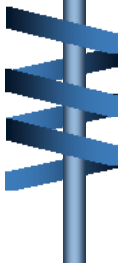
A stack of
neatly folded shirts



Stack



- An abstract data type in which elements are added and removed from only one end; a "last in, first out" (LIFO) structure.
- The items that have been present in the stack the longest are at the bottom; the most recent are at the top.
- At any time, given any two elements in a stack, one is higher than the other.

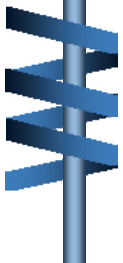


Stack



- **Operations on Stacks**

- The logical picture of the structure provides only half of the definition of an abstract data type.
- The other half consists of a set of operations that allows the user to access and manipulate the elements stored in the structure.
- Given the logical view of a stack, what kinds of operations do we need to use a stack?

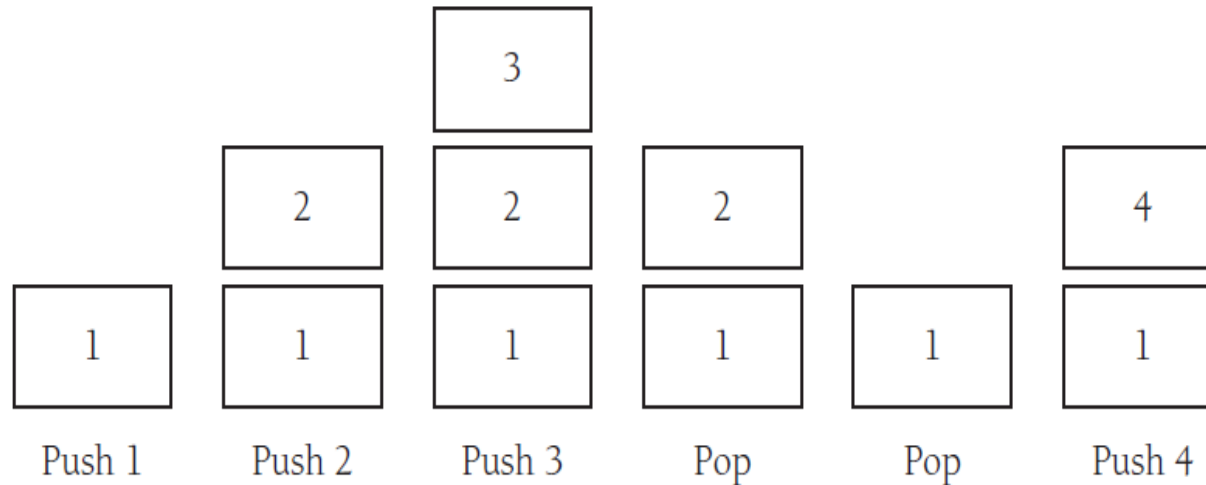


Stack



- Operations on Stacks

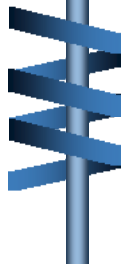
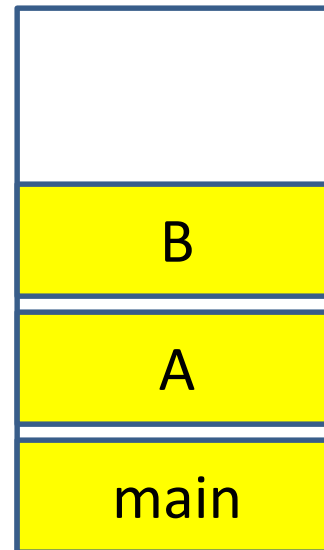
- The operation that adds an element to the top of a stack is usually called *push*
- The operation that removes the top element from the stack is referred to as *pop*



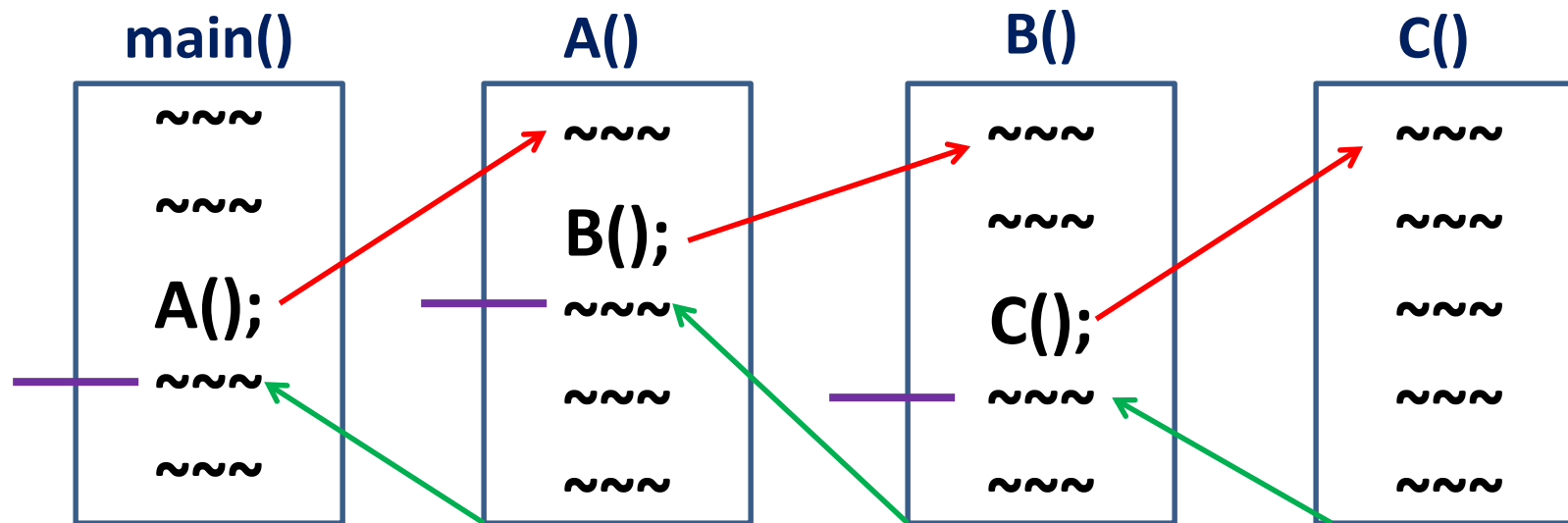
Stack : Applications



- **P**rogramming language systems typically use a stack to keep track of operation calls.
- The main program calls operation A, which in turn calls operation B, which in turn calls operation C. When C finishes, control returns to B; when B finishes, control returns to A; and so on.



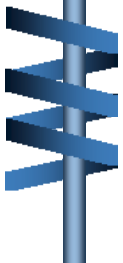
Stack : Applications



Stack : Applications



- The definition of a programming language usually consists of nested components-for example, *for* loops can contain *if-then* statements that contain *while* loops that contain *for* loops.
- As a compiler works through such nested constructs, it "saves" information about what it is currently working on in a stack. When it finishes its work on the innermost construct, the compiler can "retrieve" its previous status from the stack, and pick up where it left off.

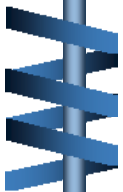


Stack : Applications



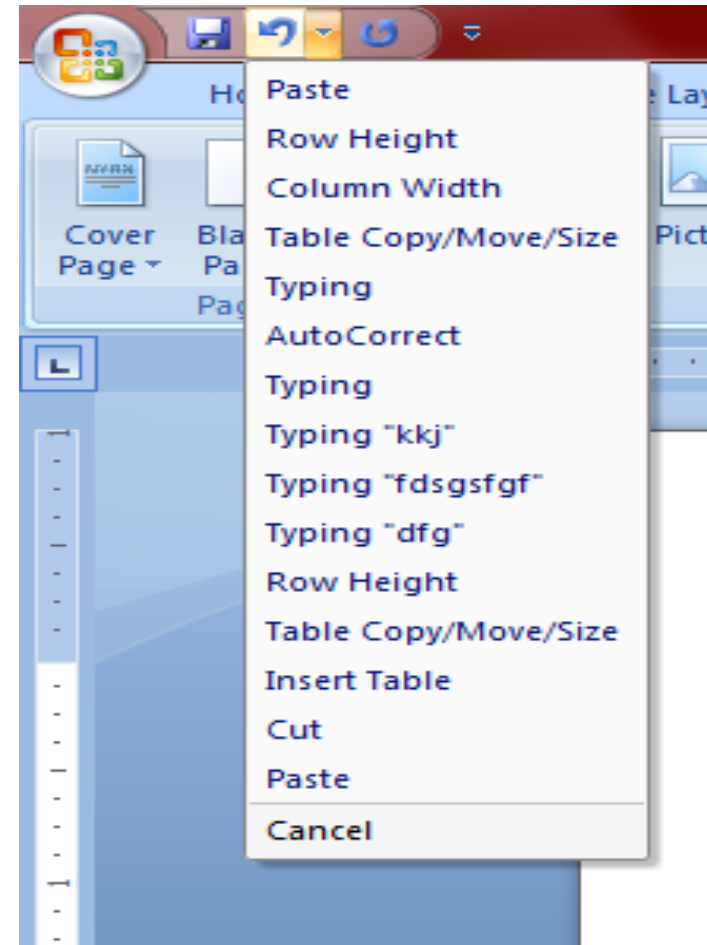
```
for (a=1;a<5;a++) {  
    for (b=1;b<5;b++) {  
        for (c=1;c<5;c++) {  
            cout<< (a+b+c) ;  
        }  
    }  
}
```

- 1** For c++
- 3** For b++
- 2** For a++



Stack : Applications

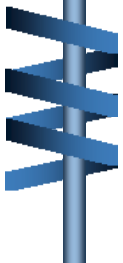
- Page-visited history in a Web browser
- Undo sequence in a text editor



Stack implementation



- **Array-based** (*Static implementation*)
 - A simple way of implementing the Stack ADT uses an array
 - However, stack size will be fixed
 - To solve this problem, growable array-based implementation could be used:
 - **Incremental strategy**: -increase the size by a constant c
 - **Doubling strategy**: -double the size



Stack implementation



- **Array-based** (*Static implementation*)

PUSH

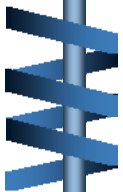
If $TOP = SIZE - 1$, then:
 Display "The stack is in overflow condition"
 Exit

else
 $TOP = TOP + 1$
 $STACK[TOP] = ITEM$
 Exit

POP

If $TOP < 0$, then
 Display "The Stack is empty"
 Exit

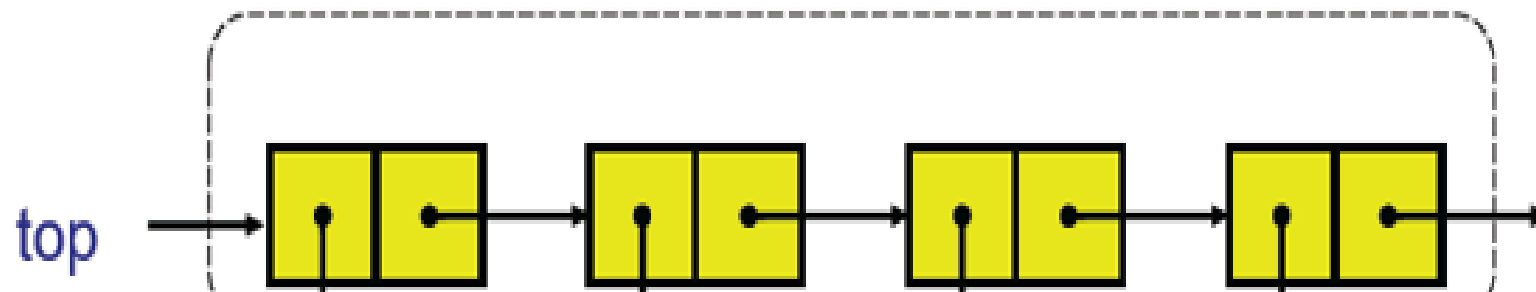
Else remove the Top most element
 $DATA = STACK[TOP]$
 $TOP = TOP - 1$
 Exit



Stack implementation



- **Linked list-based** (*Dynamic implementation*)
 - The top element is stored at the first node of the list
 - **push(Object E)**:-insert at the beginning
 - **pop()**:-delete from beginning



Stack implementation

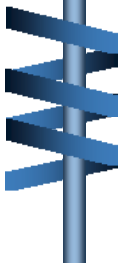


Main stack operations:

- **push**(Object E): inserts element E
- **pop**(): removes and returns the last inserted element

Auxiliary stack operations:

- **top**(): returns the last inserted element without removing it
- **size**(): returns the number of elements stored
- **isEmpty**(): a Boolean value indicating whether no elements are stored



Stack implementation

Exercise: Describe the output of the following series of stack operations

Push(8)

Push(3)

Pop()

Push(2)

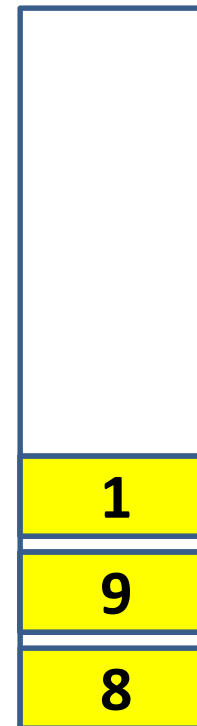
Push(5)

Pop()

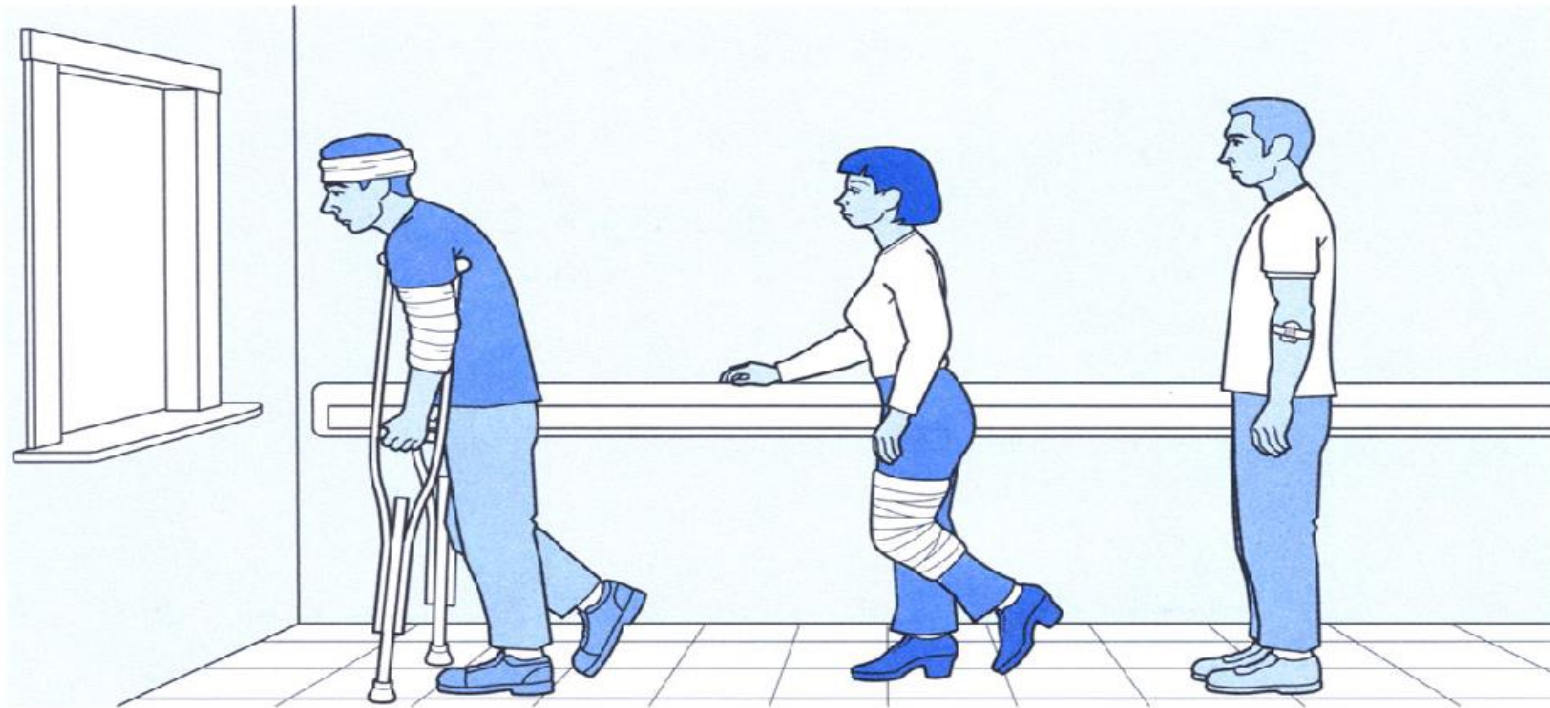
Pop()

Push(9)

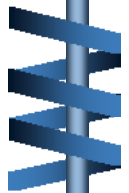
Push(1)



Queues



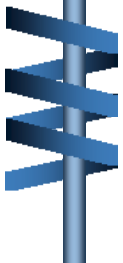
First come first served



Queues



- A queue is a data structure where data enters at the *rear* of a list and is removed from the *front* of the list
- Queues are used to store items in the order in which they occur
- Queues are an example of a first-in, first-out (FIFO) data structure
- Queues are used to order processes submitted to an operating system or a print spooler



Queues : Operations



Main operations

enqueue(object o): inserts element o at the end of the queue

dequeue(): removes and returns the element at the front of the queue

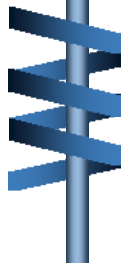
Auxiliary queue operations:

front(): returns the element at the front without removing it

size(): returns the number of elements stored

isEmpty(): returns a Boolean value indicating whether no elements are stored

- The **front** element is stored at the **head** of the list
- The **rear** element is stored at the **tail** of the list

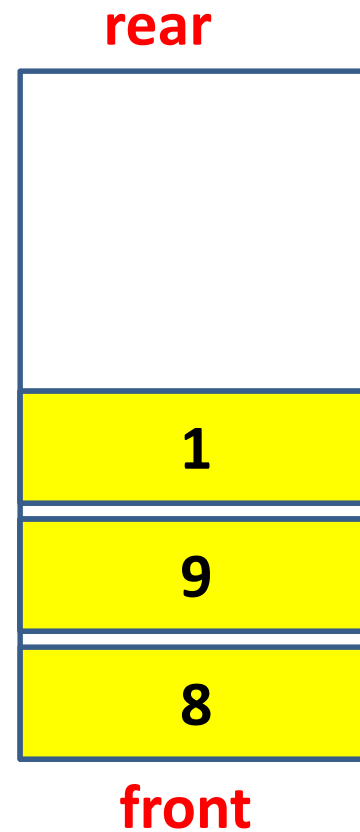


Queues : Operations



Exercise: Describe the output of the following series of queue operations

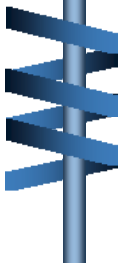
enqueue(8)
enqueue(3)
dequeue()
enqueue(2)
enqueue(5)
dequeue()
dequeue()
enqueue(9)
enqueue(1)



Queues : Application



- ✓ **Operating systems**
 - **Process queues**
 - **Message queues**
 - **Printer scheduling**
- ✓ **General scheduling applications that require a FIFO scheme**



Queues Implementation



- Array-based
 - Growable Array-based Queue
 - Incremental strategy
 - Doubling strategy

